IT342: Mobile and Cloud Computing

The 8th Lecture:
Multitasking in mobile phones

Multi-tasking types

- There are two distinct types of multitasking: *process-based* and *thread-based*.
- A ***process*** is a program that is executing.
- Process based multitasking is the feature that allows your computer to run two or more programs concurrently.
  - Example: process based multitasking enables you to run the Android Studio at the same time that you are using a text editor or visiting a web site.
- In process-based multitasking, a program is the smallest unit of code that can be dispatched by the scheduler.

Multi-tasking types

- In a *thread-based* multitasking environment, the thread is the smallest unit of dispatchable code. This means that a single program can perform two or more tasks simultaneously.
  - Example: a text editor can format text at the same time that it is printing, as long as these two actions are being performed by two separate threads.
- *Process-based* multitasking deals with the ”big picture” and *thread-based* multitasking handles the details.

Process-based multitasking VS thread-based multitasking

- Multitasking threads require less overhead than multitasking processes.
- Processes are heavy weight tasks that require their own separate address spaces.
- Context switching from one process to another is costly.
- Threads are lighter weight. They share the same address space and cooperatively share the same heavy weight process.
- Interprocess communication is expensive and limited.
- Interthread communication is inexpensive, and context switching from one thread to the next is lower in cost.

Concurrency Control: Android's Threads

- On certain occasions a single app may want to do more than one "thing" at the same time.
  - Download a large file from a website
  - Maintain a responsive UI for the user to enter data
- One solution is to have the app run those individual **concurrent** actions in separate **threads**.
- The Java Virtual-Machine provides its own *Multi-Threading* architecture

Synchronization

- When two or more threads need access to a shared resource, they need some way to ensure that the resource will be used by only one thread at a time, called ***synchronization***.

- Key to *synchronization* is the concept of the monitor. A ***monitor*** is an object that is used as a mutually exclusive lock. Only one thread can ***own*** a monitor at a given time.

- When a thread acquires a lock, it is said to have *entered* the monitor. All other threads attempting to enter the locked monitor will be suspended until the first thread *exits* the monitor.

Synchronization

- A monitor is a region of critical code executed by only one thread at the time.

```
public synchronized void methodToBeMonitored() {
   // place here your code to be lock-protected
   // (only one thread at the time!)
}

public synchronized int getGlobalVar() {
   return globalVar;
}

public synchronized void setGlobalVar(int newGlobalVar) {
   this.globalVar = newGlobalVar;
}
```

- While a thread is inside a synchronized method, all other threads that try to call it on the same instance have to wait.

Creating and Executing Threads

- The following are two strategies for creating and executing a Java Thread:
  - Create a new **Thread** instance passing to it a **Runnable** object:

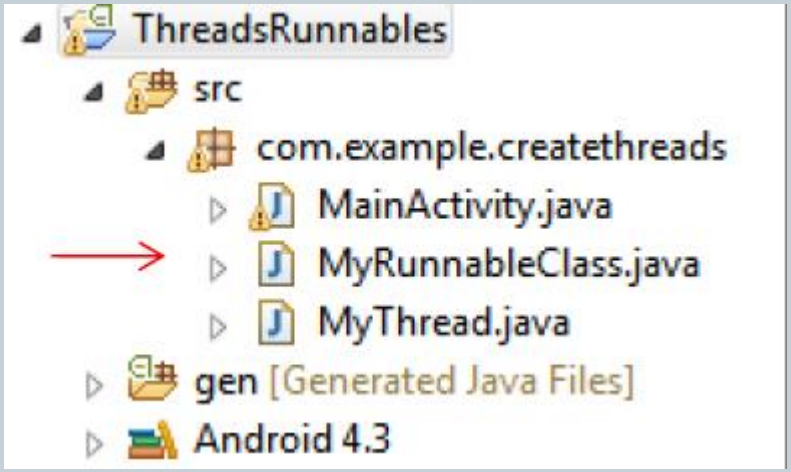
```
Runnable myRunnable1 = new MyRunnableClass();
Thread t1 = new Thread(myRunnable1);
t1.start();
```

  - Create a new custom sub-class that **extends Thread** and override its run() method:

```
MyThread t2 = new MyThread();
t2.start();
```

- In both cases, the **start()** method must be called to execute the new Thread.

A Complete Android Example: Creating Two Threads



A Complete Android Example: Creating Two Threads


```
public class MainActivity extends Activity {
@Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        Runnable myRunnable1 = new MyRunnableClass();
        Thread t1 = new Thread(myRunnable1);
        t1.start();

        MyThread t2 = new MyThread();
        t2.start();

    }//onCreate
```

A Complete Android Example: Creating Two Threads


```
public class MyRunnableClass implements Runnable {
   @Override
   public void run() {
      try {
         for (int i = 100; i < 105; i++){
            Thread.sleep(1000);
            Log.e ("t1:<<runnable>>", "runnable talking: " + i);
         }
      } catch (InterruptedException e) {
          Log.e ("t1:<<runnable>>",  e.getMessage() );
      }

   }//run

}//class
```

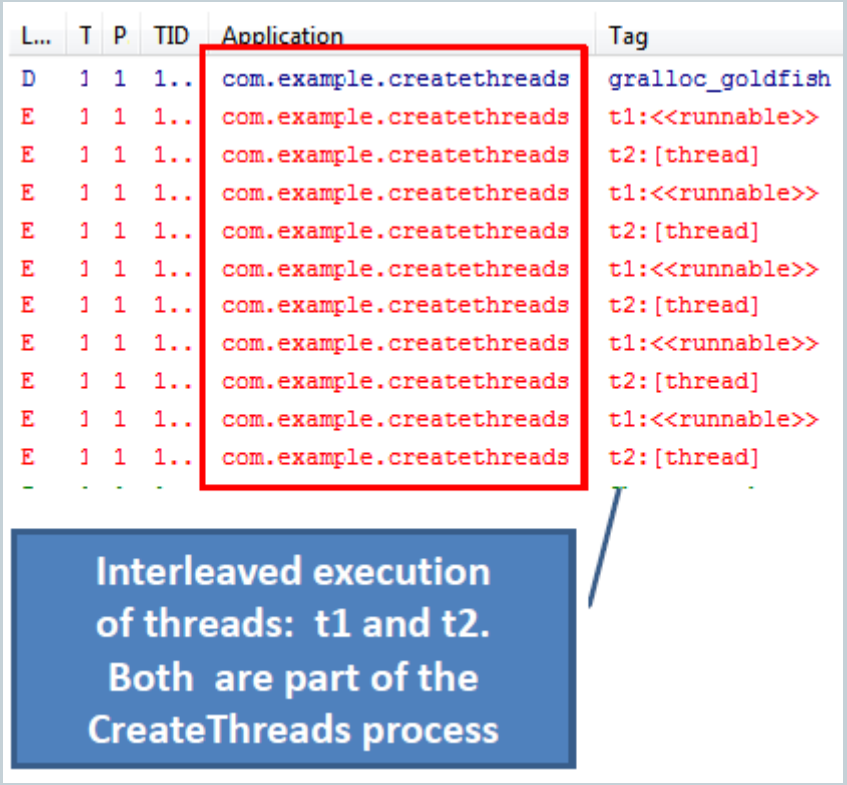
A Complete Android Example: Creating Two Threads


```
public class MyThread extends Thread{

   @Override
   public void run() {
      super.run();
      try {
         for(int i=0; i<5; i++){
            Thread.sleep(1000);
            Log.e("t2:[thread]", "Thread talking: " + i);
         }
      } catch (InterruptedException e) {
          Log.e("t2:[thread]", e.getMessage() );
      }
   }//run

}//MyThread
```

A Complete Android Example: Creating Two Threads

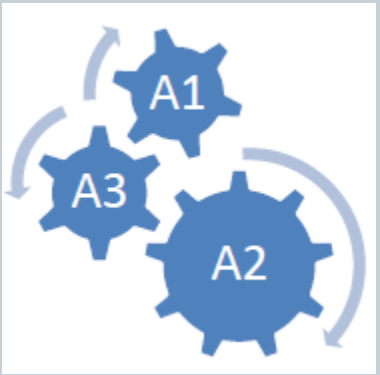


Concurrency Control

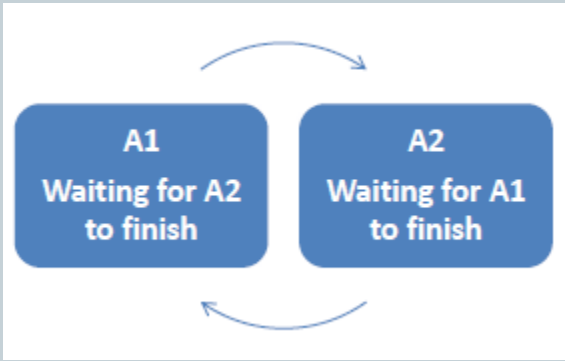
- **Advantages:**
  - Responsive applications can be easily created by placing the logic controlling the user's interaction with the UI in the application's main thread, while slow processes can be assigned to background threads.
  - A multithreaded program operates ***faster*** on computer systems that have ***multiple CPUs***. Observe that most current Android devices do provide multiple processors.

Concurrency Control

- **Disadvantages:**
  - Code tends to be more **complex.**



  - Need to detect, avoid, resolve **deadlocks.**



Concurrency Control: Strategies for Execution of Slow Activities

- **Problem:** An application may involve the use of a time-consuming operation. When the slow portion of logic executes, the other parts of the application are blocked.
- **Goal:** We want the **UI** (and perhaps other components) to be responsive to the user in spite of its heavy load.
- **Solution:** Android offers two ways for dealing with this scenario:
  - Do the slow work in a ***Background Thread***.
  - Do expensive operations in a ***Background Service***

Concurrency Control: Threads cannot touch the app's UI

- **Warning Android's background threads are not allowed to interact with the UI:**
  - All input/output involving what the user sees or supplies must be performed by the main thread.
  - A simple experiment. Add a Toast message to the run() methods implemented in Example given about creating two threads . Both should fail!

Android Services: Forms of Services

Service	Started Service	Bound Service
When to use	If the application doesn't need to communicate with the service after starting it.	If the application or other applications need to communicate with the service after starting it.
Method called to start the service	startService()	bindService()
Lifetime	Once started, a service can run in the background indefinitely, even if the component that started it is destroyed. When the operation is done, the service should stop itself.	Runs only as long as another application component is bound to it. Multiple components can bind to the service at once, but when all of them unbind, the service is destroyed.

Concurrency Control: Services cannot touch the app's UI

- **Unlike** Activities, Services don't interact with the user directly.